

Department of
Computer Science and Engineering

Daffodil International University

Department of Computer Science and Engineering

Faculty of Science and Information Technology

M.Sc. in Computer Science and Engineering

Course: Deep Learning · Fall 2026

Group Assignment 1 (CT-1)

Task 2 — CNN / Transfer Learning / Fine-tuning

Transfer Learning for Dermatoscopic Skin-Lesion Classification

Fine-tuning ImageNet-pretrained CNNs on DermaMNIST (HAM10000)

Submitted by

Member name	Student ID
Shakib Howlader	262-25-013

Submitted to

Course Instructor, Deep Learning
Department of Computer Science and Engineering
Daffodil International University

Date of submission: 26 October 2026

Codebase: `github.com/mr-shakib/dermamnist-transfer-learning`

Abstract

Skin-lesion datasets are small and dominated by benign nevi, which makes it hard to train deep networks from scratch. This report addresses **Task 2 (CNN / Transfer Learning / Fine-tuning)** by fine-tuning ImageNet-pretrained CNNs on **DermaMNIST**, the seven-class MedMNIST version of the HAM10000 dermatoscopic dataset (10,015 images, 224×224). A data analysis shows a 58.7 : 1 class imbalance, a strong colour shift from ImageNet, and a small amount of same-lesion leakage between the official splits. Under one reproducible pipeline (class-weighted loss, augmentation, validation-based model selection), we compare seven configurations: a CNN trained from scratch; feature extraction, full fine-tuning with discriminative learning rates and gradual unfreezing of ResNet-50 and EfficientNet-B0; and a class-weighting ablation. Transfer learning is decisive. Test macro-F1 rises from 48.0% (scratch) to 55.9% with a frozen ResNet-50 and to 77.2% with full fine-tuning. Gradual unfreezing (73.2%) did not help within a 12-epoch budget, and class weighting traded calibration for minority-class recall. The selected model, ResNet-50 fully fine-tuned with 8-view test-time augmentation, reaches **87.0% accuracy**, **80.1% macro-F1** (95% CI [76.6, 83.0]) and **macro-AUC 0.962** on the held-out test set, about 10 accuracy points above the official MedMNIST v2 baselines. Grad-CAM and t-SNE show lesion-focused evidence and well-separated class clusters after fine-tuning, while also pointing to hair and vignetting artefacts. All code, logs and figures are public.

Keywords: transfer learning, fine-tuning, convolutional neural networks, ResNet-50, EfficientNet, dermatoscopy, skin-lesion classification, class imbalance, DermaMNIST, HAM10000.

Contents

Abstract **i**

1 Introduction **1**

1.1 Background and motivation 1

1.2 Problem statement 1

1.3 Objectives 1

1.4 Contributions of this report 1

2 Dataset and Data Analysis **2**

2.1 Source and description 2

2.2 Class distribution and imbalance 2

2.3 Visual inspection 3

2.4 Colour statistics 4

2.5 Split integrity: duplicate and leakage audit 5

2.6 Summary of data-analysis findings 6

3 Workflow **6**

3.1 Step-by-step description 7

3.2 Experimental environment 8

4 Modelling **8**
4.1 Architectures . . . 8
4.2 Transfer-learning strategies . . . 8
4.3 Loss function and class imbalance . . . 9
4.4 Optimisation . . . 9

5 Performance Evaluation **10**
5.1 Evaluation protocol and metrics . . . 10
5.2 Training dynamics . . . 10
5.3 Main results . . . 11
5.4 Effect of the transfer strategy: gradual unfreezing . . . 12
5.5 Ablation: class-weighted loss . . . 12
5.6 Test-time augmentation . . . 13
5.7 Per-class analysis of the final model . . . 13
5.8 Calibration . . . 15
5.9 Explainability: Grad-CAM and learned representations . . . 16
5.10 Comparison with published DermaMNIST results . . . 17
5.11 Computational cost . . . 18

6 Discussion and Summary **18**
6.1 Summary of findings . . . 18
6.2 Limitations . . . 18
6.3 Ethical considerations . . . 19
6.4 Future work . . . 19
6.5 Conclusion . . . 19

7 Codebase **19**

References **20**

A Additional results **21**
A.1 Effect of test-time augmentation on every run . . . 21
A.2 Training cost . . . 22
A.3 Common hyper-parameters . . . 22

List of Figures

1 Images per class and split. The `nv` class dominates every split; `df` and `vasc` have fewer than 30 test images each. . . . 3
2 Six random training images per class (rows). Note vignetting, hairs and the strong visual similarity of `mel`, `nv` and `bkl`. . . . 4
3 Left: distribution of per-image mean luminance by class (boxes = IQR, whiskers = 1.5 IQR). Right: mean R, G, B intensity by class. . . . 5
4 Pixel-wise mean image per class (training split). Lesions are centred; pigmented classes have a brown centre (darkest and widest for `mel`), and `vasc` a purple one. . 5
5 The six test images with the closest training image by dHash (top: test, bottom: nearest train image). Pairs 1, 2 and 5 are the same lesion re-photographed; pairs 3, 4 and 6 are different lesions. . . . 6
6 End-to-end workflow. Blue: data stages; green: modelling and training; orange: evaluation and reporting. The test split is used only in the orange stages, after a checkpoint has been chosen on the validation split. . . . 7
7 Left: validation macro-F1 per epoch for all runs. Right: training and validation loss of the selected model (ResNet-50, full fine-tuning); the dashed line marks the checkpoint chosen on validation macro-F1. . . . 11
8 Test macro-F1, balanced accuracy and accuracy with 95% bootstrap confidence intervals, sorted by macro-F1. . . . 11
9 Final model on the test set: row-normalised confusion matrix (counts in brackets) and one-vs-rest ROC curves. . . . 14
10 Per-class test F1 (single view) for every run. . . . 14
11 Reliability diagram of the final model (10 bins with at least 10 images). Points above the diagonal indicate under-confidence. . . . 15
12 Grad-CAM [18] of the final ResNet-50 (last bottleneck block of `layer4`) on test images. Rows 1–2: confidently correct examples per class; rows 3–4: one misclassified example per class (single-view probabilities). . . . 16
13 t-SNE [19] of 2048-D penultimate-layer embeddings of the 2,005 test images, before (ImageNet weights) and after fine-tuning. Marker shape and colour both encode the class. . . . 17

List of Tables

1 Dataset facts. . . . 2
2 Class distribution of DermaMNIST (official splits). . . . 2
3 Near-duplicate audit with a 64-bit dHash. “Label agreement” is the share of pairs whose nearest training image has the same diagnosis; the last row is the base rate over all images. . . . 5
4 Hardware and software used for every experiment. . . . 8
5 Gradual-unfreezing schedule for ResNet-50 (12 epochs). . . . 9
6 Configuration of the seven experiments. All share the data pipeline, the loss (unless stated), AdamW, warm-up + cosine schedule, batch size 32 and seed 42. . . . 9

7 Test-set results of all runs (single view). Val F1 is the validation macro-F1 of the selected checkpoint; macro-F1 is shown with its 95% bootstrap CI. Bold: the model selected on validation. Acc. = accuracy, Bal. = balanced accuracy, both in %. . . . 11

8 Class-weighted vs unweighted cross-entropy (ResNet-50, full fine-tuning, single view). 12

9 Per-class test performance of the final model (ResNet-50 fine-tuned, 8-view TTA). 13

10 Comparison with the official MedMNIST v2 DermaMNIST benchmark [3]. The MedMNIST v2 baselines were trained on the 28×28 images (the "224" variants use 28×28 images upsampled to 224×224); our models use the native 224×224 MedMNIST+ images. 17

11 Test metrics without and with 8-view dihedral TTA (4 rotations × horizontal flip, softmax averaged). 21

12 Training cost on the GTX 1660 SUPER (FP32, batch 32). Peak memory is the PyTorch allocator peak, including the cuDNN workspace. 22

13 Hyper-parameters shared by all runs. 22

1 Introduction

1.1 Background and motivation

Skin cancer is among the most common cancers worldwide, and early recognition of malignant lesions such as melanoma and basal cell carcinoma strongly improves outcomes. Dermatoscopy (dermoscopy) is a non-invasive imaging technique that magnifies sub-surface skin structures, but reading dermatoscopic images reliably takes years of training. Deep convolutional neural networks (CNNs) have reached dermatologist-level accuracy on this problem when trained on very large image collections [5]. Most clinical settings, however, have only a few thousand labelled images, and these are heavily skewed towards benign nevi. In that low-data, imbalanced regime, *transfer learning* is the standard remedy: a network pretrained on a large generic dataset (ImageNet) is reused, and only adapted to the target domain [10, 11].

This assignment follows **Task 2 (CNN / Transfer Learning / Fine-tuning)** of the CT-1 brief: *fine-tune a pretrained CNN (e.g. ResNet, EfficientNet) on a domain-specific dataset (e.g. medical images)*, with exposure to *transfer learning, freezing/unfreezing layers and domain adaptation*. We chose medical images: the seven-class DermaMNIST benchmark [3], derived from the HAM10000 dermatoscopic collection [1]. We use the 224×224 version released with MedMNIST+ [4], so ImageNet models see images at their native input resolution.

1.2 Problem statement

Given an RGB dermatoscopic image $x \in [0, 255]^{224\times224\times3}$, predict one of seven diagnostic categories $y \in \{\texttt{akiec}, \texttt{bcc}, \texttt{bkl}, \texttt{df}, \texttt{mel}, \texttt{nv}, \texttt{vasc}\}$. A useful model must do well on the *rare* classes, including melanoma (`mel`). A model that always predicts the majority class `nv` already reaches 66.9% test accuracy, so plain accuracy alone can mislead. We therefore treat **macro-F1** as the primary metric and also report balanced accuracy, macro one-vs-rest AUC and calibration.

1.3 Objectives

1. Analyse the dataset: class balance, colour statistics and possible train/test leakage.
2. Build a reproducible PyTorch pipeline: preprocessing, augmentation, imbalance-aware training, validation-based model selection and evaluation.
3. Compare transfer-learning strategies on two pretrained backbones (ResNet-50 [8] and EfficientNet-B0 [9]) against a CNN trained from scratch: (i) *feature extraction* (frozen backbone), (ii) *full fine-tuning* with discriminative learning rates, and (iii) *gradual unfreezing* [13].
4. Measure the effect of class-weighted loss (ablation) and of test-time augmentation (TTA).
5. Explain the predictions with Grad-CAM [18] and t-SNE embeddings, and discuss limitations.

1.4 Contributions of this report

- A controlled comparison of 7 training configurations. All of them use the same data pipeline, the same seed and the same validation-based checkpoint selection; the test set is used only once, for the final numbers.
- An honest evaluation: bootstrap 95% confidence intervals, per-class metrics, calibration (ECE) and a near-duplicate audit of the official splits.
- A public, documented codebase that rebuilds every number and figure in this report with one command (Section 7).

2 Dataset and Data Analysis

2.1 Source and description

HAM10000 ("Human Against Machine with 10,000 training images") [1] is a multi-source collection of dermatoscopic images of common pigmented skin lesions. It was gathered over 20 years at the Medical University of Vienna and at a skin-cancer practice in Queensland, Australia, and it formed the training set of the ISIC 2018 challenge [2]. More than half of the diagnoses are confirmed by histopathology; the rest come from follow-up, expert consensus or in-vivo confocal microscopy. **DermaMNIST** [3] repackages the 10,015 images as a seven-class benchmark with a fixed 7 : 1 : 2 train/validation/test split. We use the **MedMNIST+ 224×224** release [4], in which the original 600×450 images are resized to 224×224 RGB (uint8) without cropping.

Table 1: Dataset facts.

Source file	`dermamnist_224.npz` (Zenodo record 10519652, 1,091 MB)
MD5 (verified)	`8974907d8e169bef5f5b96bc506ae45d` (matches `medmnist/info.py`)
Images / classes	10,015 RGB images, 7 mutually exclusive diagnostic classes
Resolution / type	224 × 224 × 3, uint8
Official split	train 7,007 / validation 1,003 / test 2,005 (stratified, image-level)
Licence	CC BY-NC 4.0 (non-commercial research and teaching use)

2.2 Class distribution and imbalance

Table 2 and Figure 1 show a severe long-tailed imbalance. Melanocytic nevi (`nv`) make up **67.0%** of the training images, while dermatofibroma (`df`) has only 80 training images (1.1%). The majority-to-minority ratio is **58.7 : 1**. Three classes are malignant or pre-malignant (`akiec`, `bcc`, `mel`); together they are only 19.5% of the data. The split is stratified, so all three splits share the same class proportions. The validation split has only 12 `df` and 14 `vasc` images, which makes per-epoch validation scores for those classes noisy.

Table 2: Class distribution of DermaMNIST (official splits).

Code	Diagnosis	Malignant	Train	Val	Test	Total	Share
`akiec`	Actinic keratoses / intraepithelial carcinoma	pre-malignant	228	33	66	327	3.3%
`bcc`	Basal cell carcinoma	yes	359	52	103	514	5.1%
`bkl`	Benign keratosis-like lesions	no	769	110	220	1,099	11.0%
`df`	Dermatofibroma	no	80	12	23	115	1.1%
`mel`	Melanoma	yes	779	111	223	1,113	11.1%
`nv`	Melanocytic nevi	no	4,693	671	1,341	6,705	67.0%
`vasc`	Vascular lesions	no	99	14	29	142	1.4%
	Total		**7,007**	**1,003**	**2,005**	**10,015**	100%

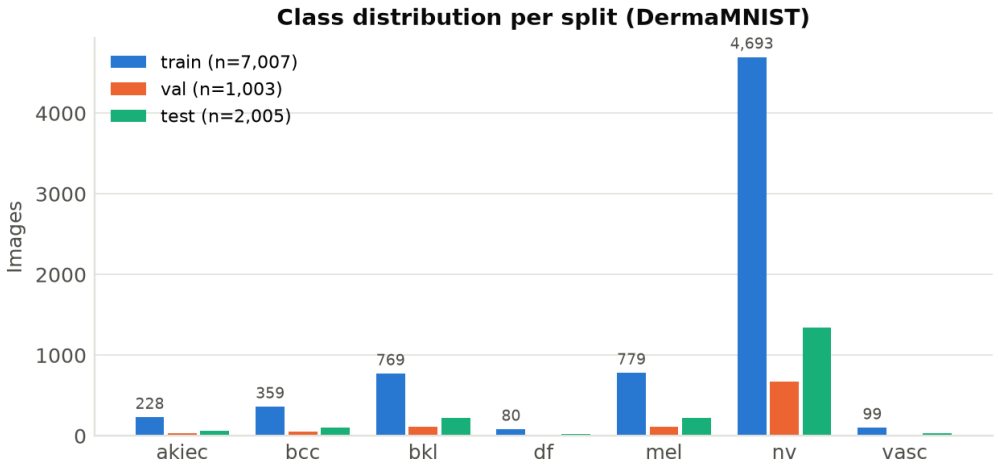


Figure 1: Images per class and split. The `nv` class dominates every split; `df` and `vasc` have fewer than 30 test images each.

Consequences for modelling. (i) A classifier that always answers `nv` already scores 66.9% test accuracy but a macro-F1 of only 0.11, so accuracy cannot be the selection metric; we select on validation *macro-F1*. (ii) Rare classes need explicit help, so we use inverse-frequency class weights in the loss (ablated in Section 5.5). (iii) With 23–29 test images, per-class scores for `df` and `vasc` carry wide uncertainty, so we report bootstrap confidence intervals.

2.3 Visual inspection

Figure 2 shows random training images. They vary a lot in illumination, zoom and skin tone. Typical acquisition artefacts are dark circular *vignetting* from the dermatoscope lens, hairs and occasional reference objects in the field of view (e.g. in one `df` image). The classes overlap visually: many `mel`, `nv` and `bkl` lesions are brown, roughly round pigmented blobs, and they differ mainly in fine texture, border irregularity and colour variegation. Vascular lesions (`vasc`) stand out by their red/purple hue. The lesion is almost always centred, and orientation is arbitrary. This motivates flip and 90°-rotation augmentation, which leaves the label unchanged.

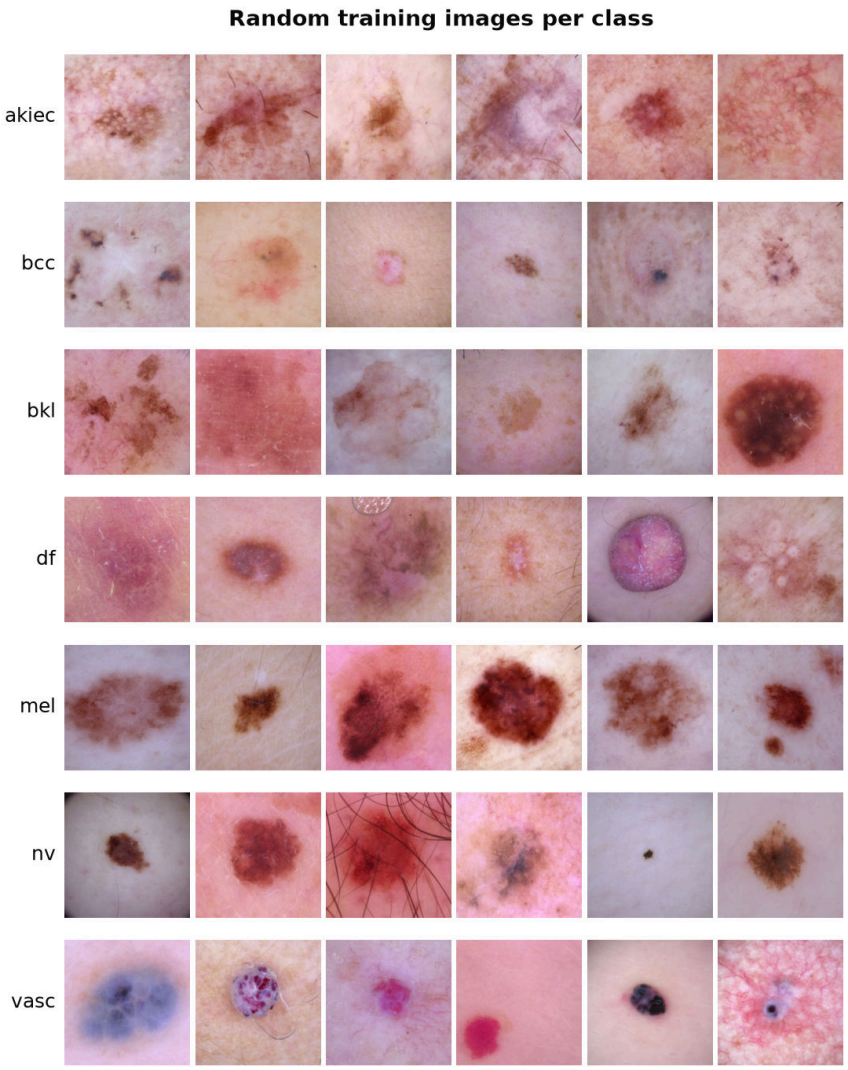


Figure 2: Six random training images per class (rows). Note vignetting, hairs and the strong visual similarity of `mel`, `nv` and `bkl`.

2.4 Colour statistics

Over the training split, the per-channel mean (R, G, B) is (0.763, 0.538, 0.562) and the standard deviation is (0.137, 0.159, 0.177). The images are red-dominant skin tones, quite unlike the ImageNet statistics (0.485, 0.456, 0.406), which is one aspect of the *domain shift* that fine-tuning has to bridge. We still normalise with the ImageNet statistics, because the pretrained filters expect inputs on that scale. Figure 3 shows that colour carries class information. Melanoma and benign keratoses are the darkest classes (median luminance 0.584 and 0.592), while basal cell carcinoma is the brightest (0.668); `vasc` and `bcc` have the highest blue component. The per-class mean images (Figure 4) confirm that lesions are centred. The pigmented classes (`mel`, `nv`, `bkl`) have a dark brown centre, and the `mel` blob is the darkest and widest. `akiec`, `bcc` and `df` are only faintly pink, being less pigmented, and `vasc` has a distinctly purple centre. Because colour is informative but also depends on the acquisition device, we use only *mild* colour jitter (Section 3). Strong jitter could erase real diagnostic cues.

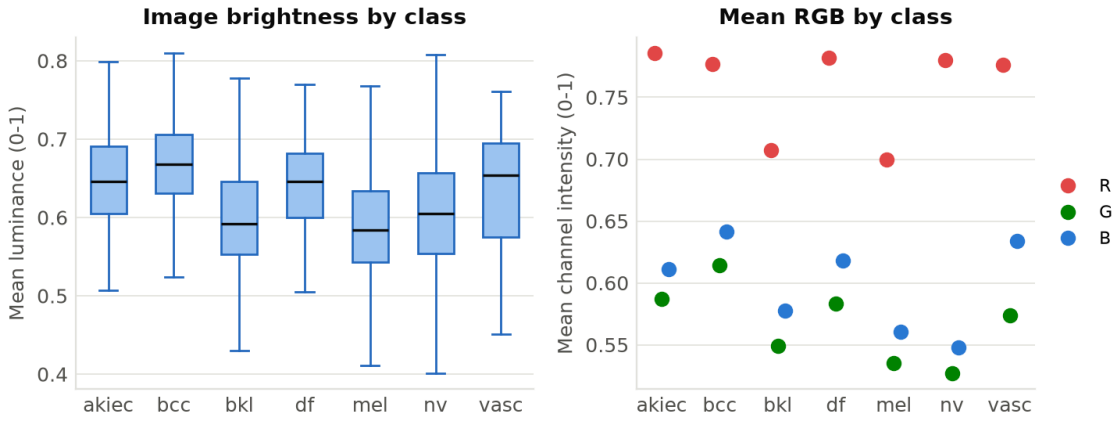


Figure 3: Left: distribution of per-image mean luminance by class (boxes = IQR, whiskers = 1.5 IQR). Right: mean R, G, B intensity by class.



Figure 4: Pixel-wise mean image per class (training split). Lesions are centred; pigmented classes have a brown centre (darkest and widest for `mel`), and `vasc` a purple one.

2.5 Split integrity: duplicate and leakage audit

HAM10000 often contains *several images of the same lesion* [1], and the DermaMNIST split is made per image, not per lesion. Lesion identifiers are not distributed with DermaMNIST, so we audited the splits directly in two ways:

1. **Exact duplicates** (MD5 of raw pixels): **none** of the validation or test images is a byte-identical copy of a training image.
2. **Near duplicates**: a 64-bit difference hash (dHash) of each image, then the Hamming distance to the nearest training image (Table 3).

Table 3: Near-duplicate audit with a 64-bit dHash. "Label agreement" is the share of pairs whose nearest training image has the same diagnosis; the last row is the base rate over all images.

	Validation (n = 1,003)		Test (n = 2,005)	
Distance to nearest train image	pairs	label agreement	pairs	label agreement
$d = 0$	3	100%	3	100%
$d \leq 2$	15	73.3%	29	69.0%
$d \leq 4$	63	60.3%	122	70.5%
any d (base rate)	1,003	56.0%	2,005	59.2%

The pairs at $d = 0$ always share the label. Figure 5 confirms that several of the closest pairs are the same lesion photographed twice, with a slightly different shift or zoom. For $d \leq 4$, though, label agreement (60–71%) is barely above the base rate (56–59%). Coarse dHash therefore mostly finds

look-alike lesions (round brown blobs on pink skin), not re-photographs. Our conclusion: a *small* amount of same-lesion leakage does exist between the official splits, so test scores may be slightly optimistic. It cannot be measured exactly without lesion IDs. We keep the official split so that our numbers stay comparable with published DermaMNIST results, and we list this as a limitation (Section 6.2).

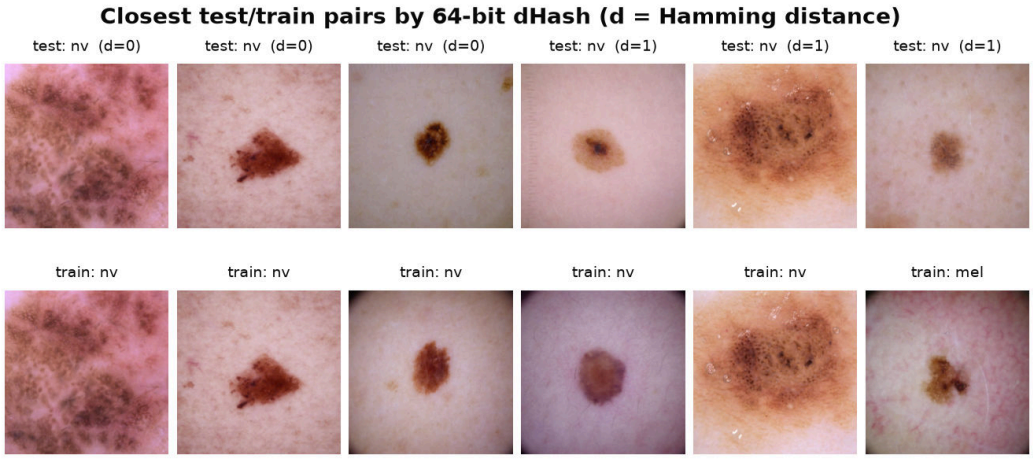


Figure 5: The six test images with the closest training image by dHash (top: test, bottom: nearest train image). Pairs 1, 2 and 5 are the same lesion re-photographed; pairs 3, 4 and 6 are different lesions.

2.6 Summary of data-analysis findings

- Severe imbalance (58.7 : 1). We use macro-F1 for model selection and class-weighted loss for training.
- Arbitrary lesion orientation. We augment with flips and 90° rotations; the test-time version of this becomes 8-view TTA.
- Colour is informative but depends on the device. We use only mild colour jitter.
- Strong domain shift from ImageNet (colour statistics, texture-dominated content). Full fine-tuning should beat frozen features (tested in Section 5).
- Small, partly measurable same-lesion leakage in the official splits. We keep it in mind when interpreting results.

3 Workflow

Figure 6 shows the end-to-end pipeline. Each box maps to one script in the codebase, so the whole study can be re-run with `bash scripts/run_all.sh`.

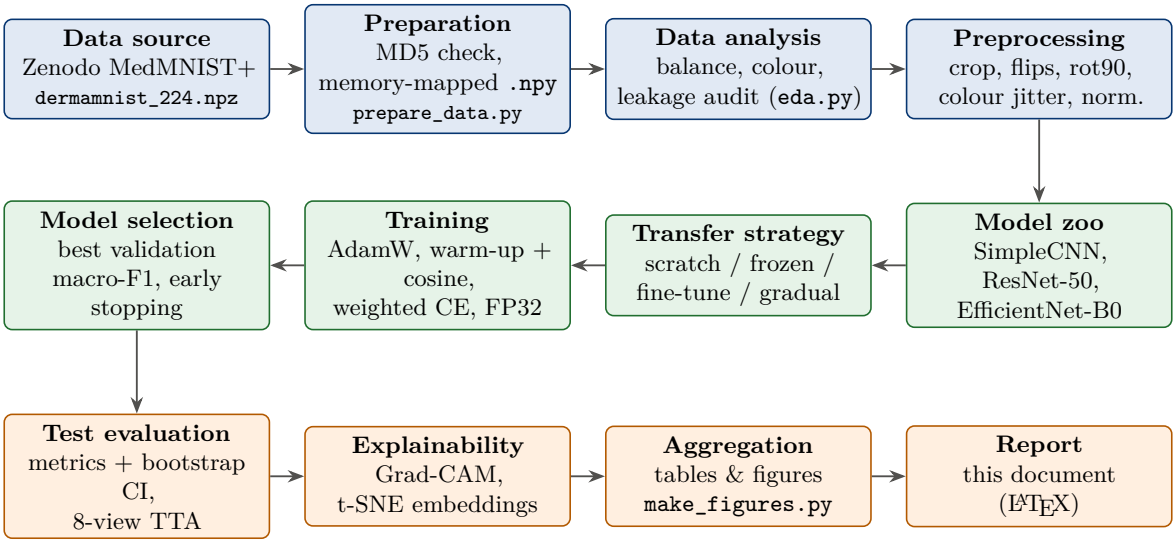


Figure 6: End-to-end workflow. Blue: data stages; green: modelling and training; orange: evaluation and reporting. The test split is used only in the orange stages, after a checkpoint has been chosen on the validation split.

3.1 Step-by-step description

1. **Acquisition.** The 224×224 DermaMNIST archive (1,091 MB) is downloaded from the MedMNIST+ Zenodo record and checked with an MD5 checksum.
2. **Preparation.** The three official splits are written as separate `.npy` files and *memory-mapped*, so the 1.5 GB uint8 tensor is never fully loaded into RAM. This matters on the 16 GB development machine.
3. **Data analysis** (Section 2). Class counts, colour statistics, mean images, and exact and near-duplicate checks between splits.
4. **Preprocessing and augmentation.** Training images go through `RandomResizedCrop`(224, scale 0.75–1.0), horizontal and vertical flips, random multiples of 90° rotation (dermatoscopic images have no canonical orientation), mild colour jitter (brightness/contrast 0.2, saturation 0.15, hue 0.02) and ImageNet mean/std normalisation. Validation and test images are only normalised.
5. **Training** (Section 4). Every configuration uses AdamW, a one-epoch linear warm-up followed by cosine decay, FP32 arithmetic (Section 4 explains why not FP16), gradient clipping at norm 5 and class-weighted cross-entropy with label smoothing 0.05.
6. **Model selection.** After each epoch the model is scored on the validation split. The checkpoint with the highest *validation macro-F1* is kept, and training stops early if there is no improvement for 6 epochs (8 for the scratch baseline).
7. **Evaluation** (Section 5). The selected checkpoint is scored once on the test split, with and without 8-view dihedral TTA. Metrics come with 1,000-resample bootstrap confidence intervals.
8. **Explainability.** Grad-CAM heat-maps and t-SNE plots of the penultimate-layer embeddings, for the best model.

3.2 Experimental environment

Table 4: Hardware and software used for every experiment.

GPU	NVIDIA GeForce GTX 1660 SUPER, 6 GB GDDR6 (Turing, no tensor cores)
CPU / RAM	AMD Ryzen 5 5500 (6 cores / 12 threads) / 16 GB
OS	Ubuntu 26.04 LTS, NVIDIA driver 595.91
Software	Python 3.12, PyTorch [20] 2.14.1 (CUDA 12.6), torchvision 0.29.1, scikit-learn 1.9.1
Seeding	global seed 42 for Python, NumPy and PyTorch (cuDNN autotuning on, so not bit-exact)

4 Modelling

4.1 Architectures

Baseline: SimpleCNN trained from scratch. A VGG-style network [7] with five blocks. Each block has two 3×3 convolution–BatchNorm–ReLU layers followed by 2×2 max-pooling, with widths 16-32-64-128-256. Global average pooling, dropout ($p = 0.3$) and a linear layer produce the 7 logits. It has **1.18 M** parameters, all randomly initialised (PyTorch default Kaiming-uniform). This baseline shows what is achievable *without* transfer learning.

ResNet-50 [8]. A 50-layer residual network with bottleneck blocks arranged in four stages (`layer1`–`layer4`, output stride 32), followed by global average pooling. We use the torchvision `IMAGENET1K_V2` weights (80.86% ImageNet top-1, 4.09 GFLOPs). The 1000-way classifier is replaced by dropout ($p = 0.3$) and a $2048 \to 7$ linear layer, giving **23.52 M** parameters.

EfficientNet-B0 [9]. A compound-scaled network built from inverted-residual MBConv blocks with squeeze-and-excitation and SiLU activations. We use torchvision `IMAGENET1K_V1` weights (77.69% ImageNet top-1, 0.39 GFLOPs, about 10× cheaper than ResNet-50). The head is again dropout followed by a $1280 \to 7$ linear layer, giving **4.02 M** parameters.

4.2 Transfer-learning strategies

Let f_θ be the pretrained backbone and g_ϕ the new classification head. We compare:

1. **Scratch** (baseline only): θ, ϕ randomly initialised, all trained with learning rate 10^{-3}.
2. **Feature extraction (frozen backbone).** θ is fixed at its ImageNet values and BatchNorm is kept in inference mode (running statistics are not updated). Only g_ϕ is trained, with learning rate 10^{-3}. This is the cheapest option: no gradients flow through the backbone.
3. **Full fine-tuning with discriminative learning rates.** All parameters are trained, but the backbone uses a 10× smaller learning rate ($\eta_\theta = 10^{-4}$) than the randomly initialised head ($\eta_\phi = 10^{-3}$). The new head can adapt fast while the pretrained features are only gently adjusted, which limits catastrophic forgetting.
4. **Gradual unfreezing** (ULMFiT-style [13]). Training starts with the head only, and one backbone stage is unfrozen every two epochs, from the output side inwards (Table 5). Each stage nearer the input gets half the learning rate of the stage above it, because early layers hold generic edge and colour detectors that need little change [10].

Table 5: Gradual-unfreezing schedule for ResNet-50 (12 epochs).

Epochs	Trainable modules	Peak LR of newest stage	Trainable params
1–2	head	10^{-3} (head)	14 K
3–4	+ `layer4`	1.0×10^{-4}	14.98 M
5–6	+ `layer3`	5.0×10^{-5}	22.08 M
7–8	+ `layer2`	2.5×10^{-5}	23.30 M
9–12	+ stem, `layer1`	1.25×10^{-5}	23.52 M

4.3 Loss function and class imbalance

We minimise class-weighted cross-entropy with label smoothing [14] $\varepsilon = 0.05$:

$$\mathcal{L} = \frac{1}{\sum_i w_{y_i}} \sum_i \Big[(1-\varepsilon)\, w_{y_i} \big(-\log p_{i,y_i} \big) + \frac{\varepsilon}{K} \sum_{c=1}^{K} w_c \big(-\log p_{i,c} \big) \Big], \tag{1}$$

where $p_{i,c}$ is the softmax probability of class c for image i and $K = 7$ (this is exactly PyTorch's `CrossEntropyLoss(weight=w, label_smoothing=`ε`)`). The class weights are the *square root* of the inverse class frequency, rescaled to mean 1:

$$w_c \propto \Big(\frac{N}{K\, n_c} \Big)^{1/2}. \tag{2}$$

Full inverse-frequency weighting would weight `df` 59 times more than `nv`; with only 80 `df` images, that tends to overfit the rare classes. The square root is a milder compromise. The resulting weights are `akiec` 1.11, `bcc` 0.88, `bkl` 0.60, `df` 1.87, `mel` 0.60, `nv` 0.24, `vasc` 1.68. Section 5.5 compares this with unweighted cross-entropy.

4.4 Optimisation

We use AdamW [15] ($\beta = (0.9, 0.999)$, weight decay 10^{-4}) with a batch size of 32. The learning rate rises linearly over the first epoch (warm-up) and then follows a cosine decay to zero [16]:

$$\eta_t = \eta_{\max} \cdot \tfrac{1}{2} \Big(1 + \cos\big(\pi\, \tfrac{t - t_w}{T - t_w} \big) \Big), \quad t \geq t_w. \tag{3}$$

Gradients are clipped to an ℓ_2 norm of 5. We trained in FP32. Our benchmark showed FP16 autocast to be 3.3× *slower* on this GPU (34 vs 112 img/s for ResNet-50), because the GTX 16-series has no tensor cores. Mixed precision is still available in the code (`-amp`) for tensor-core GPUs.

Table 6: Configuration of the seven experiments. All share the data pipeline, the loss (unless stated), AdamW, warm-up + cosine schedule, batch size 32 and seed 42.

ID	Run name (code)	Backbone	Strategy	Ep.	LR head	LR backbone	Loss
E1	`scratch_cnn`	SimpleCNN	scratch	30	10^{-3}	(all layers 10^{-3})	weighted CE
E2	`resnet50_frozen`	ResNet-50	frozen	10	10^{-3}	frozen	weighted CE
E3	`effnetb0_frozen`	EffNet-B0	frozen	10	10^{-3}	frozen	weighted CE
E4	`resnet50_finetune`	ResNet-50	fine-tune	12	10^{-3}	10^{-4}	weighted CE
E5	`effnetb0_finetune`	EffNet-B0	fine-tune	12	10^{-3}	10^{-4}	weighted CE
E6	`resnet50_gradual`	ResNet-50	gradual	12	10^{-3}	$10^{-4} \rightarrow 1.25{\cdot}10^{-5}$	weighted CE
E7	`resnet50_finetune_noweights`	ResNet-50	fine-tune	12	10^{-3}	10^{-4}	plain CE

Model selection and early stopping. After every epoch we compute validation macro-F1 and keep the checkpoint with the best value. Training stops when there has been no improvement for 6 epochs (8 for E1). The test split is never used for any decision; it is evaluated once per run, at the end.

5 Performance Evaluation

5.1 Evaluation protocol and metrics

Each run is evaluated once on the 2,005-image test split, using the checkpoint chosen on validation macro-F1. With TP_c, FP_c, FN_c counted one-vs-rest for each class c:

- **Accuracy**: fraction of correct predictions. This is the official MedMNIST metric, but it is dominated by `nv`.
- **Balanced accuracy**: mean per-class recall, $\frac{1}{K}\sum_c \mathrm{TP}_c/(\mathrm{TP}_c + \mathrm{FN}_c)$.
- **Macro-F1** (primary): unweighted mean of per-class $F_1 = 2\,\mathrm{P}_c\mathrm{R}_c/(\mathrm{P}_c + \mathrm{R}_c)$. Every class counts equally, including melanoma and the rare classes.
- **Macro AUC**: mean one-vs-rest area under the ROC curve. It is threshold-free and is the second official MedMNIST metric.
- **Expected calibration error (ECE)** [17]: with 15 equal-width confidence bins B_m, $\mathrm{ECE} = \sum_m \frac{|B_m|}{n}\,\big|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\big|$. Lower is better. This matters when predicted probabilities are used for triage.
- **95% confidence intervals**: percentile bootstrap over 1,000 resamples of the test set.

Test-time augmentation (TTA) averages the softmax outputs over the 8 dihedral transforms of each image (4 rotations $\times$ {identity, horizontal flip}). These are exactly the symmetries the model saw during training. Whether the best model uses TTA is also decided on the validation split.

5.2 Training dynamics

Figure 7 (left) shows validation macro-F1 per epoch for all seven runs. Three regimes are visible:

- **Scratch.** The SimpleCNN learns slowly and noisily, climbing from 0.20 to its best value of 0.49 at epoch 19, then plateauing. Early stopping ended it at epoch 27.
- **Frozen backbones** plateau within about five epochs at 0.55 (ResNet-50) and 0.59 (EfficientNet-B0). A linear head on fixed ImageNet features has little capacity to adapt to dermatoscopic texture.
- **Fine-tuned backbones** pass 0.70 by epoch 3–4 and keep improving, with ResNet-50 reaching 0.795 at epoch 11.

Epoch-to-epoch swings of ± 0.03–0.05 are expected. The validation split has only 12 `df` and 14 `vasc` images, so a single image changes that class's recall by 7–8 points.

The loss curves of the selected model (Figure 7, right) show no overfitting within 12 epochs: the validation loss keeps falling until the selected epoch. The training loss stays *above* the validation loss because it is measured under augmentation, dropout, label smoothing and class weighting. The validation loss is plain cross-entropy on clean images, so the two curves are not on the same scale.

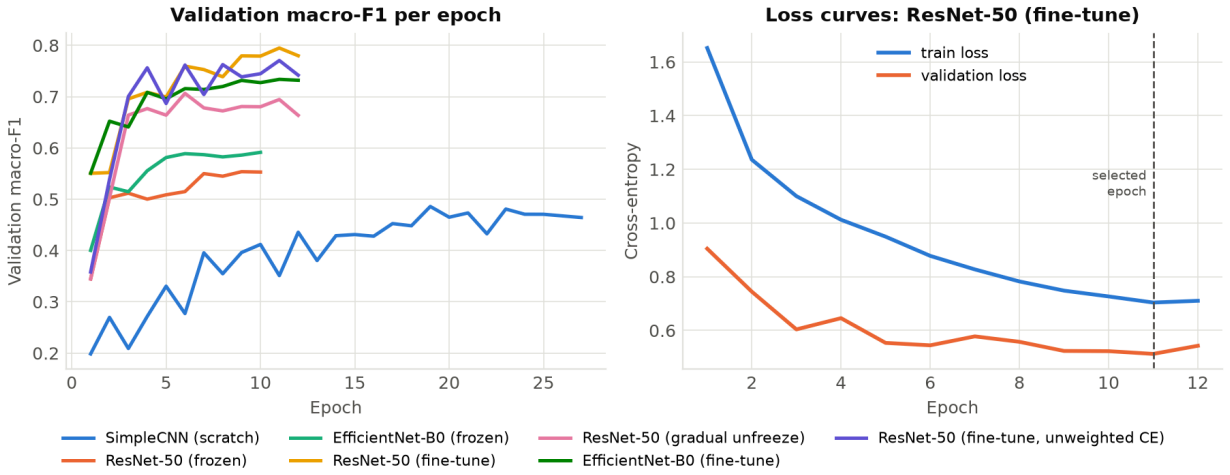


Figure 7: Left: validation macro-F1 per epoch for all runs. Right: training and validation loss of the selected model (ResNet-50, full fine-tuning); the dashed line marks the checkpoint chosen on validation macro-F1.

5.3 Main results

Table 7 and Figure 8 summarise the test results (single view, i.e. without TTA, so every run is compared on equal terms).

Table 7: Test-set results of all runs (single view). Val F1 is the validation macro-F1 of the selected checkpoint; macro-F1 is shown with its 95% bootstrap CI. Bold: the model selected on validation. Acc. = accuracy, Bal. = balanced accuracy, both in %.

Run	Params (M)	Trainable (M)	Best ep. / run	Val F1 (%)	Acc. (%)	Bal. (%)	Macro-F1 [95% CI] (%)	AUC	ECE
SimpleCNN (scratch)	1.18	1.18	19/27	48.6	73.6	49.4	48.0 [44.6, 51.0]	0.908	0.163
ResNet-50 (frozen)	23.52	0.01	9/10	55.4	76.5	56.6	55.9 [51.3, 60.3]	0.920	0.198
EfficientNet-B0 (frozen)	4.02	0.01	10/10	59.1	75.3	63.5	59.4 [55.2, 63.0]	0.929	0.171
ResNet-50 (fine-tune)	**23.52**	**23.52**	**11/12**	**79.5**	**86.1**	**76.3**	**77.2 [73.4, 80.5]**	**0.955**	**0.084**
ResNet-50 (gradual unfreeze)	23.52	23.52	6/12	70.6	83.0	74.1	73.2 [69.3, 76.4]	0.958	0.133
EfficientNet-B0 (fine-tune)	4.02	4.02	11/12	73.4	84.8	76.0	72.9 [69.0, 76.3]	0.959	0.103
ResNet-50 (fine-tune, unweighted CE)	23.52	23.52	11/12	77.1	86.9	71.8	75.7 [71.7, 79.4]	0.971	0.024
Majority class (always `nv`)					66.9	14.3	11.5	0.500	–

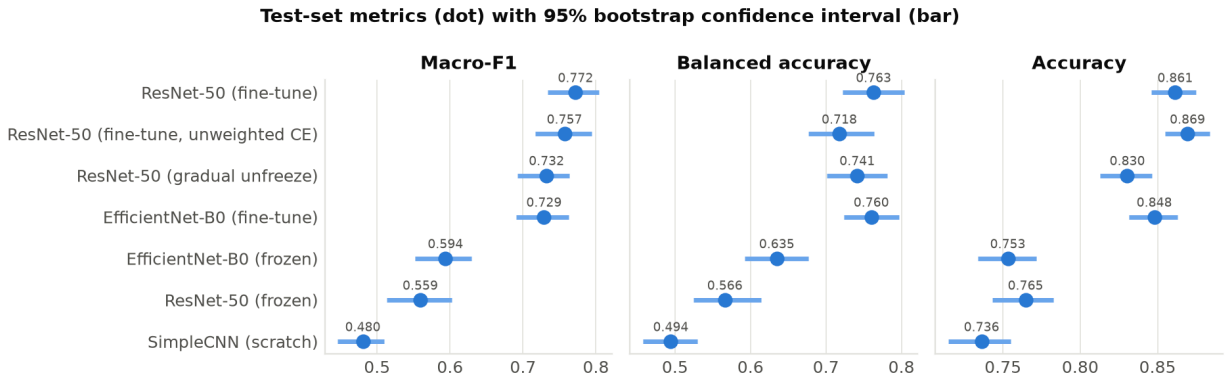


Figure 8: Test macro-F1, balanced accuracy and accuracy with 95% bootstrap confidence intervals, sorted by macro-F1.

Finding 1: transfer learning is decisive. The from-scratch SimpleCNN reaches 73.6% accuracy, only 6.7 points above the trivial majority classifier, and a macro-F1 of just 48.0%. It never correctly detects a single dermatofibroma (`df` F1 = 0.0). Fine-tuning an ImageNet ResNet-50 raises macro-F1 to 77.2% (+29.2 points) and accuracy to 86.1%. The 95% CIs, [73.4, 80.5] against [44.6, 51.0], are far apart, so the gain is not a sampling artefact. With only 80–100 training images for the rarest classes, features learned from 1.28 M natural images [6] give a far better starting point than random initialisation.

Finding 2: frozen features are not enough; the domain shift has to be bridged. Feature extraction improves on scratch training (macro-F1 55.9% for ResNet-50, 59.4% for EfficientNet-B0), but full fine-tuning adds another **+21.3** (ResNet-50) and **+13.5** (EfficientNet-B0) points. This matches the colour and texture shift found in Section 2. Dermatoscopic images look nothing like ImageNet photographs, so the mid- and high-level filters must change, not just the classifier on top. The same pattern has been reported for other medical-imaging transfer problems [12].

Finding 3: ResNet-50 is the most accurate, EfficientNet-B0 the most efficient. When frozen, EfficientNet-B0 transfers better (59.4% vs 55.9% macro-F1). After fine-tuning, ResNet-50 leads on macro-F1 (77.2% vs 72.9%) and accuracy (86.1% vs 84.8%), while EfficientNet-B0 has a slightly higher AUC (0.959 vs 0.955). EfficientNet-B0 does this with 5.9× fewer parameters, 10× fewer FLOPs and less than half the training time (6.6 vs 13.8 min). It is the better choice under a tight compute or deployment budget. The two macro-F1 CIs overlap ([69.0, 76.3] vs [73.4, 80.5]), so the ResNet-50 lead is suggestive rather than conclusive.

5.4 Effect of the transfer strategy: gradual unfreezing

Gradual unfreezing reached 73.2% macro-F1, below full fine-tuning (77.2%). Its best validation epoch was epoch 6, the end of the phase in which only `layer3`, `layer4` and the head were trainable. Validation F1 did not improve after `layer2` and the stem were released. Two effects explain this:

1. the shared cosine schedule had already decayed the learning rate by then, so the lower stages, which start with $\leq 2.5 \times 10^{-5}$, barely move;
2. in a 12-epoch budget, half of the epochs are spent with most of the network frozen.

The main purpose of gradual unfreezing is to protect pretrained features from large early gradients. In full fine-tuning, warm-up and discriminative learning rates already provide that protection. Gradual unfreezing still has advantages: it is about 30% cheaper (9.7 vs 13.8 min) and has the highest single-view AUC among the class-weighted ResNet runs (0.958). With a longer budget, or with a separate schedule restarted for each newly unfrozen stage, the gap would likely narrow.

5.5 Ablation: class-weighted loss

Removing the class weights (E7 vs E4, everything else identical) shows a clear trade-off:

Table 8: Class-weighted vs unweighted cross-entropy (ResNet-50, full fine-tuning, single view).

Loss	Acc. (%)	Bal. acc. (%)	Macro-F1 (%)	AUC	ECE	F1 `df`	F1 `vasc`
weighted ($\sqrt{\text{inv. freq.}}$)	86.1	**76.3**	**77.2**	0.955	0.084	**74.4**	**89.3**
unweighted	**86.9**	71.8	75.7	**0.971**	**0.024**	68.4	84.0

Class weighting raises balanced accuracy by 4.5 points and macro-F1 by 1.5 points. The gain comes from the two rarest classes (`df` +6.0 and `vasc` +5.3 F1 points). The cost is a little accuracy on the

majority classes (overall accuracy −0.8, `nv` F1 93.3 vs 94.3) and much worse calibration (ECE 0.084 vs 0.024). Weighting deliberately shifts probability mass towards rare classes, so the raw softmax outputs no longer estimate true class frequencies. The unweighted model has the best AUC of all runs (0.971), so its *ranking* of images is excellent. Its default arg-max decision rule simply favours `nv`. In practice: if calibrated risk scores are needed, train unweighted and tune per-class thresholds on validation; if balanced detection with a plain arg-max is needed, use weighting. The macro-F1 CIs overlap ([73.4, 80.5] vs [71.7, 79.4]), so the macro-F1 difference is small relative to test-set noise.

5.6 Test-time augmentation

Averaging over the 8 dihedral views (Appendix, Table 11) improves accuracy (+0.7 to +1.9 points) and macro-AUC (+0.003 to +0.010) for *every* run. It improves macro-F1 for six of the seven runs (+1.9 to +3.3 points; frozen EfficientNet-B0 is essentially unchanged at −0.1), at 8× the inference cost. For the selected model, TTA also raised validation macro-F1 (0.795 → 0.810), so the final model uses TTA, a decision made on validation only. Its final test performance is:

Final model: ResNet-50, full fine-tuning, 8-view TTA (test set, n = 2,005)		
Accuracy	**87.0%**	95% CI [85.5, 88.4]
Balanced accuracy	**79.2%**	95% CI [75.3, 82.9]
Macro-F1	**80.1%**	95% CI [76.6, 83.0]
Macro AUC	**0.962**	95% CI [0.948, 0.974]
Cohen's κ / ECE	0.761 / 0.102	

5.7 Per-class analysis of the final model

Table 9: Per-class test performance of the final model (ResNet-50 fine-tuned, 8-view TTA).

Class	Support	Precision (%)	Recall (%)	F1 (%)	AUC
`akiec`	66	93.8	45.5	61.2	0.863
`bcc`	103	79.8	88.3	83.9	0.989
`bkl`	220	72.8	84.1	78.1	0.979
`df`	23	86.4	82.6	84.4	0.971
`mel`	223	61.0	80.7	69.5	0.959
`nv`	1341	96.1	90.6	93.3	0.977
`vasc`	29	100.0	82.8	90.6	1.000

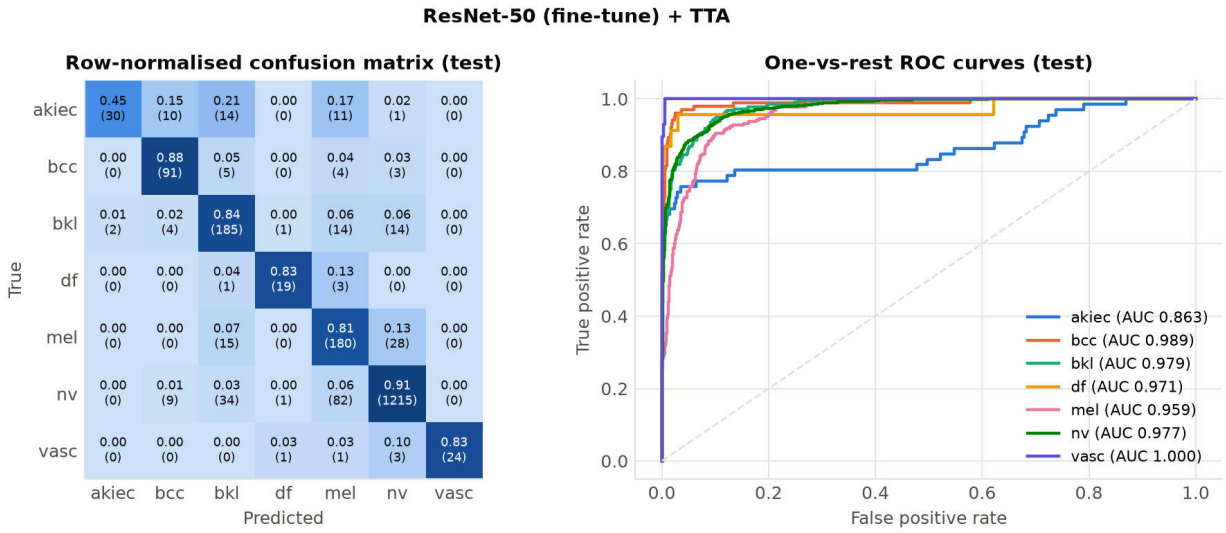


Figure 9: Final model on the test set: row-normalised confusion matrix (counts in brackets) and one-vs-rest ROC curves.

Table 9 and Figure 9 show where the model succeeds and fails.

- **Strong classes.** `nv` (F1 93.3%), `vasc` (90.6%, with 100% precision), `df` (84.4%) and `bcc` (83.9%). The visually distinctive `vasc` and `bcc` classes reach AUC 1.000 and 0.989.
- **Melanoma** has recall 80.7% and precision 61.0%. Of the 223 melanomas, 28 are called `nv` and 15 `bkl`. In the other direction, 82 nevi (6.1% of all nevi) are flagged as melanoma, which accounts for most of its false positives. For a screening aid, this direction of error (over-calling melanoma rather than missing it) is the safer one.
- **Actinic keratoses (`akiec`)** are the weakest class. When the model predicts `akiec` it is almost always right (precision 93.8%), but it finds only 45.5% of them. Missed cases go to `bkl` (14), `mel` (11) and `bcc` (10). `akiec` and `bkl` are both scaly, keratinocytic lesions, and pigmented actinic keratoses can mimic melanoma. Its lower AUC (0.863) shows that the class is hard to separate at any threshold, not only at the arg-max.

Figure 10 compares per-class F1 across all runs. Fine-tuning improves every class. The largest gains are on `bcc` (+0.29 over frozen ResNet-50), `df` (+0.36) and `vasc` (+0.30), the classes with the most distinctive local structure.

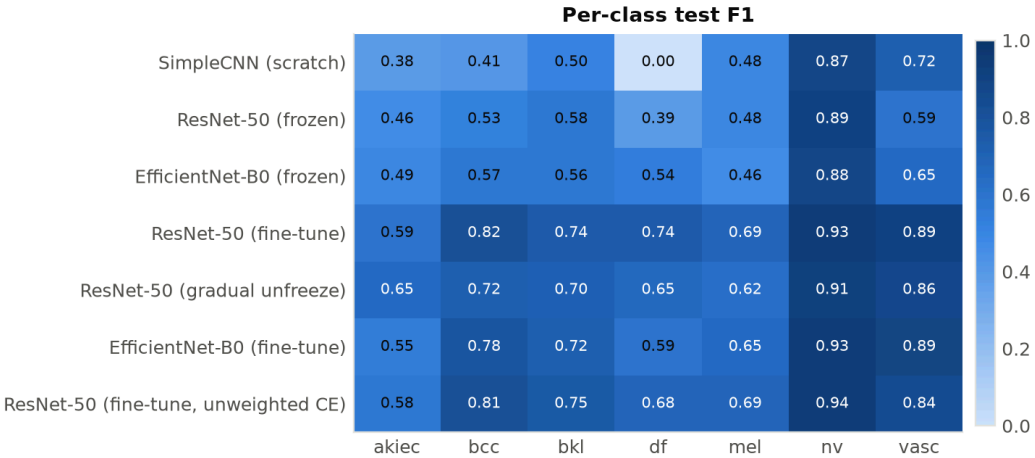


Figure 10: Per-class test F1 (single view) for every run.

5.8 Calibration

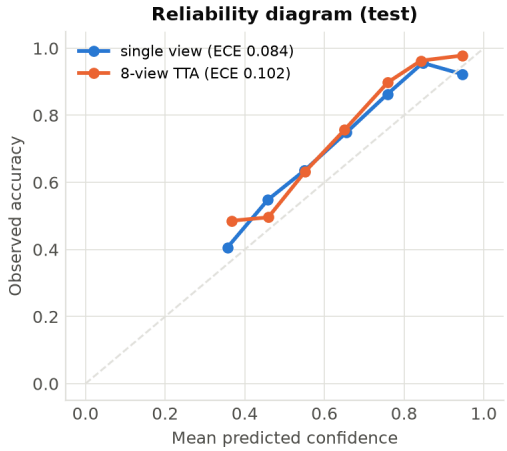


Figure 11: Reliability diagram of the final model (10 bins with at least 10 images). Points above the diagonal indicate under-confidence.

The reliability diagram (Figure 11) lies *above* the diagonal: predictions made with 55–65% confidence are correct 63–75% of the time, so the model is *under*-confident (ECE 0.084 single view, 0.102 with TTA). The causes are label smoothing (which caps the target probability at 0.957), class weighting (Section 5.5) and, for TTA, averaging over views, which further softens the outputs. Ranking quality (AUC) is unaffected. If calibrated probabilities are needed, one temperature parameter fitted on the validation split (temperature scaling [17]) would correct this without changing any prediction.

5.9 Explainability: Grad-CAM and learned representations

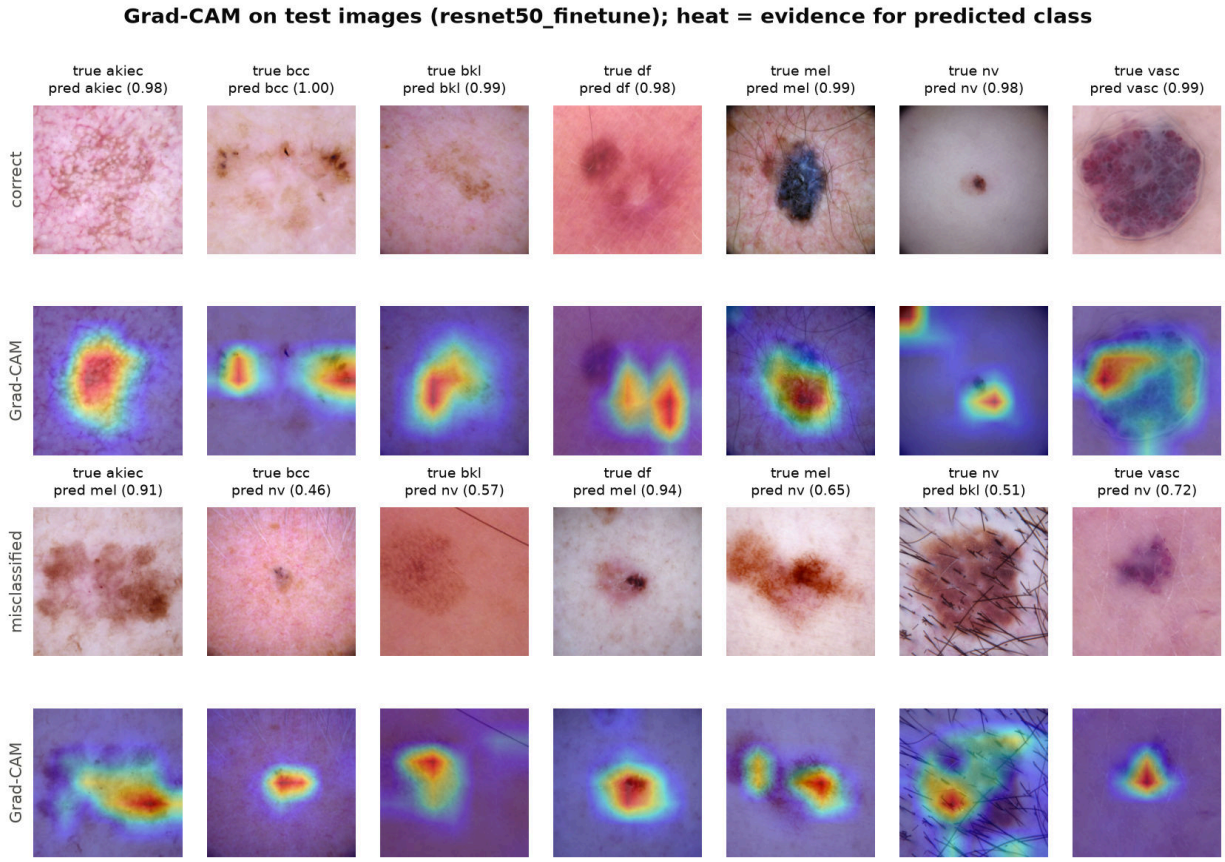


Figure 12: Grad-CAM [18] of the final ResNet-50 (last bottleneck block of `layer4`) on test images. Rows 1–2: confidently correct examples per class; rows 3–4: one misclassified example per class (single-view probabilities).

Grad-CAM (Figure 12) shows that, for most correct predictions, the evidence lies on the lesion itself:

- the scattered pigmented foci of the basal cell carcinoma;
- the blue-black centre of the melanoma;
- the red-purple lobules of the vascular lesion;
- the central lesion area of the actinic keratosis.

Two observations call for caution. First, in the correctly classified nevus, a strong activation also sits in the *dark vignetted corner* of the image, which points to a possible acquisition-artefact shortcut. Second, for the nevus misclassified as `bkl`, the heat is spread over the *hairs* covering the lesion. Both point to hair removal and vignetting-aware cropping as useful preprocessing. Five of the seven misclassified examples are low-confidence, genuinely ambiguous cases (0.46–0.72). The two exceptions are a pigmented `akiec` (0.91) and a `df` (0.94), both predicted as melanoma.

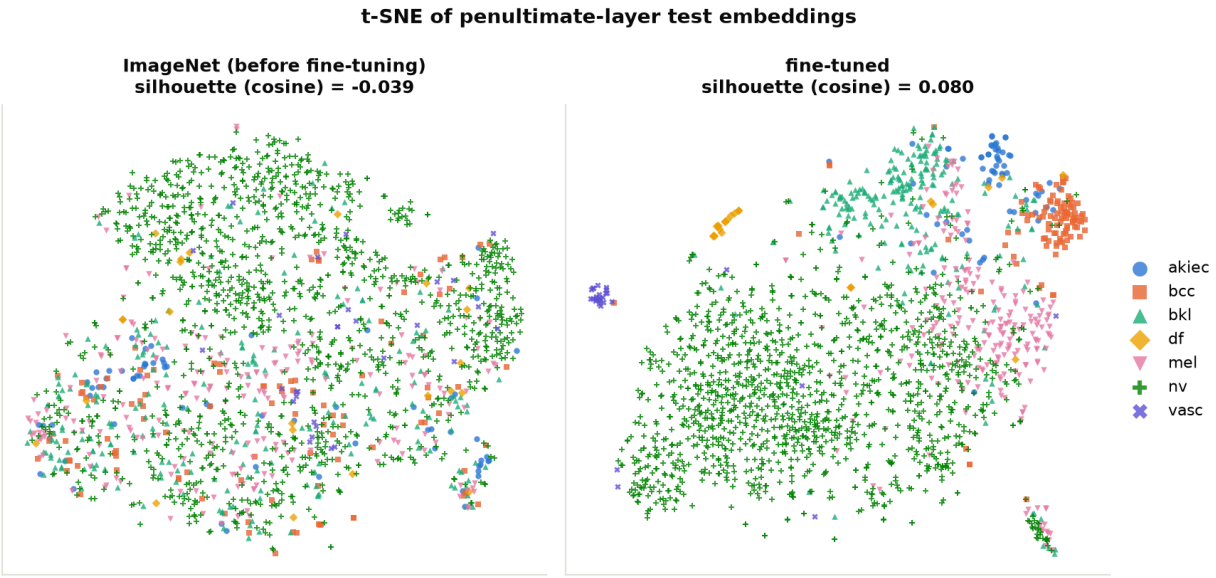


Figure 13: t-SNE [19] of 2048-D penultimate-layer embeddings of the 2,005 test images, before (ImageNet weights) and after fine-tuning. Marker shape and colour both encode the class.

Embeddings. Figure 13 shows what fine-tuning changes inside the network. With ImageNet weights, all classes are intermixed (cosine silhouette -0.039). After fine-tuning, `vasc`, `df`, `bcc`, `akiec` and `bkl` form compact, separate clusters, and the silhouette becomes positive (0.080). Melanoma occupies a region that borders and overlaps the large nevus cloud, which matches the `mel`↔`nv` confusions that dominate the error analysis.

5.10 Comparison with published DermaMNIST results

Table 10: Comparison with the official MedMNIST v2 DermaMNIST benchmark [3]. The MedMNIST v2 baselines were trained on the 28×28 images (the "224" variants use 28×28 images upsampled to 224×224); our models use the native 224×224 MedMNIST+ images.

Source	Method	Test ACC	Test AUC
MedMNIST v2	ResNet-18 (28) / ResNet-18 (224)	0.735 / 0.754	0.917 / 0.920
MedMNIST v2	ResNet-50 (28) / ResNet-50 (224)	0.735 / 0.731	0.913 / 0.912
MedMNIST v2	auto-sklearn / AutoKeras	0.719 / 0.749	0.902 / 0.915
MedMNIST v2	Google AutoML Vision	0.768	0.914
This work	SimpleCNN, scratch (E1)	0.736	0.908
This work	EfficientNet-B0, fine-tuned (E5)	0.848	0.959
This work	ResNet-50, fine-tuned (E4)	0.861	0.955
This work	**ResNet-50, fine-tuned + TTA (final)**	**0.870**	**0.962**

Our scratch baseline lands in the same range as the MedMNIST v2 baselines (0.72–0.77 ACC), which is a useful sanity check of the pipeline. The fine-tuned models improve on the best published baseline by about 10 accuracy points and 0.04 AUC. This gain combines two factors, pretraining and native-resolution input, and the comparison does not separate them. For the effect of pretraining alone, Table 7 (identical data and resolution) is the controlled comparison.

5.11 Computational cost

All seven runs together took 59 minutes of training on the GTX 1660 SUPER (Appendix, Table 12). A full ResNet-50 fine-tuning epoch takes 66 s and peaks at 3.9 GB of GPU memory. EfficientNet-B0 needs 32 s per epoch, and feature extraction only 10–20 s, because no gradients flow through the backbone.

6 Discussion and Summary

6.1 Summary of findings

This assignment studied transfer learning for seven-class dermatoscopic skin-lesion classification on DermaMNIST (HAM10000, 224×224). Seven controlled experiments led to five conclusions:

1. **Pretraining matters most.** Fine-tuning an ImageNet ResNet-50 lifts test macro-F1 from 48.0% (CNN trained from scratch) to 77.2%, and accuracy from 73.6% to 86.1%.
2. **Fine-tune, do not freeze.** Frozen ImageNet features reach only 55.9–59.4% macro-F1. The dermatoscopic domain is far enough from ImageNet that the backbone itself must adapt; discriminative learning rates with warm-up make this stable.
3. **Gradual unfreezing did not help** in a 12-epoch budget (73.2% vs 77.2%), although it is cheaper and ranks well (AUC 0.958).
4. **Class weighting trades accuracy and calibration for minority-class recall.** Balanced accuracy goes up 4.5 points, `df/vasc` F1 up 5–6 points, and accuracy down 0.8 points. ECE worsens from 0.024 to 0.084.
5. **8-view TTA is a reliable, free-to-train gain.** It improves accuracy and AUC for every model.

The final model, **ResNet-50 with full fine-tuning and 8-view TTA**, selected on validation only, reaches **87.0% accuracy**, **79.2% balanced accuracy**, **80.1% macro-F1** (95% CI [76.6, 83.0]) and **macro-AUC 0.962** on the held-out test set. That is about 10 accuracy points above the best result in the official MedMNIST v2 benchmark table. It detects 80.7% of melanomas, and its errors are concentrated in the clinically hard `mel/nv` and `akiec/bkl` pairs. Grad-CAM shows that its evidence usually lies on the lesion itself.

6.2 Limitations

- **Same-lesion leakage.** The official split is image-level. Our audit (Section 2.5) found re-photographed training lesions in the test set, so the absolute scores are probably slightly optimistic. A lesion-level split needs the HAM10000 metadata, which DermaMNIST does not ship.
- **Single seed and single split.** Each configuration was trained once (seed 42). The bootstrap CIs capture test-set sampling noise but not training variance. Several of the closer comparisons (ResNet-50 vs EfficientNet-B0, weighted vs unweighted) overlap within their CIs and should be repeated over 3–5 seeds before drawing firm conclusions.
- **Limited tuning budget.** Learning rates, epochs and the unfreezing schedule were set from common practice, not searched, because of the 6 GB GPU. Gradual unfreezing in particular might do better with a longer schedule.
- **Under-confident probabilities.** The final model's raw probabilities are under-confident (ECE 0.102) and should be temperature-scaled before being read as risk scores.

- **Dataset scope.** HAM10000 comes from two centres and mostly fair skin types. Performance on darker skin, on other dermatoscopes, or on clinical (non-dermatoscopic) photographs is untested.

6.3 Ethical considerations

This is a course project on a public research dataset (CC BY-NC 4.0). It is *not* a medical device and must not be used for diagnosis. In any clinical use, the most costly error is a missed melanoma, so recall on the malignant classes and calibrated uncertainty matter more than overall accuracy. The model would also need validation across skin types to avoid unequal performance between patient groups.

6.4 Future work

- Re-split by lesion using HAM10000 metadata and re-evaluate, to remove the leakage.
- Repeat every run over 5 seeds and report mean ± standard deviation; tune per-class decision thresholds and the temperature on the validation split.
- Add hair-removal and vignette-aware preprocessing, which Grad-CAM suggests is needed.
- Compare stronger backbones (ConvNeXt, Vision Transformers, dermatology-specific foundation models) and larger input sizes using the same pipeline [4].
- Test external generalisation on ISIC 2019/2020 images.

6.5 Conclusion

For a small, imbalanced medical-imaging dataset, ImageNet transfer learning with *full* fine-tuning and discriminative learning rates turned a weak from-scratch baseline (48.0% macro-F1) into a strong classifier (80.1% macro-F1, 87.0% accuracy with TTA). This was done on a single 6 GB consumer GPU in about an hour of total training. The comparison also shows that the details matter: whether to freeze or fine-tune, how to handle class imbalance, and how to evaluate (macro-F1, confidence intervals, calibration, leakage checks). These choices change both the numbers and the conclusions one should draw from them.

7 Codebase

Repository: `https://github.com/mr-shakib/dermamnist-transfer-learning`

The repository contains all source code, the experiment script, the per-epoch training logs, evaluation outputs (JSON), every figure, and the LaTeX source of this report. Model checkpoints and the dataset are excluded because of their size; `scripts/run_all.sh` downloads the data and regenerates both.

Repository layout

```
dermamnist-transfer-learning/
|-- src/
| |-- common.py paths, class names, memory-mapped Dataset, augmentation, class weights
| |-- prepare_data.py MD5-checked .npz -> per-split .npy conversion
| |-- eda.py data analysis + figures + duplicate/leakage audit
| |-- models.py SimpleCNN, ResNet-50, EfficientNet-B0, stage split, embeddings
```

```
| |-- train.py scratch / frozen / finetune / gradual training with model selection
| |-- metrics.py accuracy, balanced acc., macro-F1, AUC, ECE, bootstrap CIs
| |-- evaluate.py test evaluation with and without 8-view TTA
| |-- make_figures.py comparison figures and LaTeX tables
| |-- gradcam.py Grad-CAM and t-SNE for the selected model
| |-- report_numbers.py numbers quoted in this report, as LaTeX macros
| `-- plot_style.py shared colour-blind-safe figure style
|-- scripts/run_all.sh reproduces every experiment, table and figure
|-- results/ eda.json, summary.json, runs/<name>/{config,history,eval}, figures/, logs/

|-- report/ LaTeX source and compiled PDF of this report
`-- requirements.txt
```

Reproducing the results

```
git clone https://github.com/mr-shakib/dermamnist-transfer-learning.git
cd dermamnist-transfer-learning
python3.12 -m venv .venv
.venv/bin/pip install torch torchvision --index-url https://download.pytorch.org/whl/
    cu126
.venv/bin/pip install -r requirements.txt
bash scripts/run_all.sh # about 70 min on a GTX 1660 SUPER
cd report && latexmk -pdf main.tex
```

References

[1] P. Tschandl, C. Rosendahl, and H. Kittler, "The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions," *Scientific Data*, vol. 5, art. 180161, 2018.

[2] N. Codella *et al.*, "Skin lesion analysis toward melanoma detection 2018: A challenge hosted by the International Skin Imaging Collaboration (ISIC)," arXiv:1902.03368, 2019.

[3] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister, and B. Ni, "MedMNIST v2 – A large-scale lightweight benchmark for 2D and 3D biomedical image classification," *Scientific Data*, vol. 10, art. 41, 2023.

[4] S. Doerrich, F. Di Salvo, J. Brockmann, and C. Ledig, "Rethinking model prototyping through the MedMNIST+ dataset collection," *Scientific Reports*, vol. 15, art. 7669, 2025, doi:10.1038/s41598-025-92156-9.

[5] A. Esteva, B. Kuprel, R. A. Novoa, J. Ko, S. M. Swetter, H. M. Blau, and S. Thrun, "Dermatologist-level classification of skin cancer with deep neural networks," *Nature*, vol. 542, pp. 115–118, 2017.

[6] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. CVPR*, 2009.

[7] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. ICLR*, 2015.

[8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, pp. 770–778, 2016.

[9] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. ICML*, pp. 6105–6114, 2019.

[10] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Advances in NeurIPS*, 2014.

[11] S. Kornblith, J. Shlens, and Q. V. Le, "Do better ImageNet models transfer better?" in *Proc. CVPR*, 2019.

[12] M. Raghu, C. Zhang, J. Kleinberg, and S. Bengio, "Transfusion: Understanding transfer learning for medical imaging," in *Advances in NeurIPS*, 2019.

[13] J. Howard and S. Ruder, "Universal language model fine-tuning for text classification," in *Proc. ACL*, 2018.

[14] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception architecture for computer vision," in *Proc. CVPR*, 2016.

[15] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.

[16] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. ICLR*, 2017.

[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, 2017.

[18] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. ICCV*, 2017.

[19] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.

[20] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in NeurIPS*, 2019.

A Additional results

A.1 Effect of test-time augmentation on every run

Table 11: Test metrics without and with 8-view dihedral TTA (4 rotations × horizontal flip, softmax averaged).

	Accuracy (%)		Macro-F1 (%)		Macro-AUC	
Run	single	TTA	single	TTA	single	TTA
SimpleCNN (scratch)	73.6	74.8	48.0	50.6	0.908	0.911
ResNet-50 (frozen)	76.5	77.2	55.9	57.8	0.920	0.926
EfficientNet-B0 (frozen)	75.3	76.7	59.4	59.3	0.929	0.935
ResNet-50 (fine-tune)	86.1	87.0	77.2	80.1	0.955	0.962
ResNet-50 (gradual unfreeze)	83.0	84.2	73.2	76.5	0.958	0.968
EfficientNet-B0 (fine-tune)	84.8	86.1	72.9	75.6	0.959	0.966
ResNet-50 (fine-tune, unweighted CE)	86.9	88.8	75.7	78.5	0.971	0.979

A.2 Training cost

Table 12: Training cost on the GTX 1660 SUPER (FP32, batch 32). Peak memory is the PyTorch allocator peak, including the cuDNN workspace.

Run	s / epoch	train time (min)	peak GPU mem. (GB)
SimpleCNN (scratch)	19	8.9	1.33
ResNet-50 (frozen)	20	3.9	2.77
EfficientNet-B0 (frozen)	10	1.9	1.18
ResNet-50 (fine-tune)	66	13.8	3.86
ResNet-50 (gradual unfreeze)	45	9.7	3.01
EfficientNet-B0 (fine-tune)	32	6.6	3.66
ResNet-50 (fine-tune, unweighted CE)	67	14.1	3.86

A.3 Common hyper-parameters

Table 13: Hyper-parameters shared by all runs.

Input	224×224 RGB, ImageNet mean/std normalisation
Train augmentation	RandomResizedCrop (scale 0.75–1, ratio 0.9–1.1), H/V flip, rot90, ColorJitter (0.2, 0.2, 0.15, 0.02)
Optimiser	AdamW, $\beta = (0.9, 0.999)$, weight decay 10^{-4}, gradient clip 5.0
Schedule	1-epoch linear warm-up, cosine decay to 0 (per step)
Batch size	32 (train), 64 (evaluation)
Loss	cross-entropy, label smoothing 0.05, $\sqrt{\text{inverse-frequency}}$ class weights
Regularisation	dropout 0.3 before the classifier, weight decay, augmentation
Selection	best validation macro-F1; early-stopping patience 6 (8 for scratch)
Seed	42